

Custom keyed Hash Function via Merkle-Damgard Construction

CSCI361-project2
spring 2025-2026



Presented by:

Aya khalifeh

Abed Al Kareem Khaled Hussein



Objective

- ❑ Design a keyed hash function (MAC)
- ❑ Based on Merkle–Damgård construction
- ❑ Produces a fixed 8-bit output
- ❑ Ensures:
 - ❑ Data integrity
 - ❑ Authentication using a secret key



Background concept

- Hash Function: Maps input $\rightarrow$ fixed-size output
- Keyed Hash (MAC): Uses a secret key for security
- Merkle–Damgård:
 - Processes data in blocks
 - Uses chaining values (CV)
- Compression Function:
 - Reduces input to fixed size

Algorithm overview

- ❑ Input: Message + Secret Key
- ❑ Steps:
 - ❑ 1. Pad message to 16 bits
 - ❑ 2. Split into 16-bit block
 - ❑ 3. Initialize CV = Key
 - ❑ 4. Apply compression (2 rounds)
 - ❑ 5. Output final 8-bit hash



Padding design

- Append 4-bit message length
- Add zeros between message and length
- Final size = 16 bits

Example:

- Message: 10101100 (8 bits)
- Length: 1000
- Padding: 0000

Final:

- 10101100 0000 1000
- [10101100] → [0000] [1000]
- **Message Zeros Length**



Block format

- Total block size: 16 bits
- Split into:
 - Left (L): first 8 bits
 - Right (R): last 8 bits

Example:

- $L = 10101100$
- $R = 00001000$



Key integration

- ❑ Secret key is 8 bits
- ❑ Used as:
 - ❑ Initial Value ($IV = Key$)
 - ❑ Inside compression rounds
- ❑ Ensures:
 - ❑ Hash cannot be computed without key

Compression function

Input:

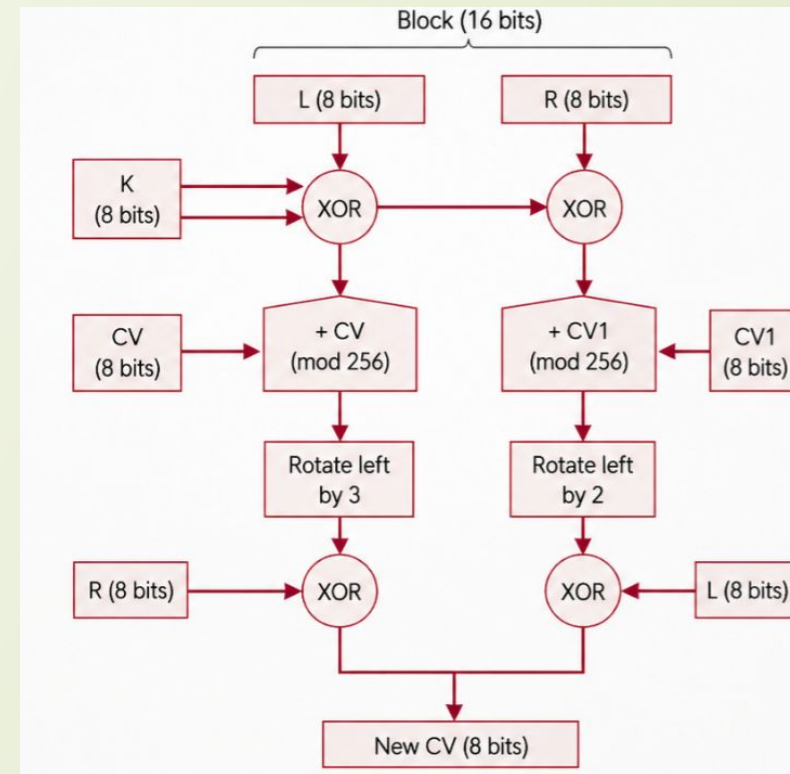
- CV (8 bits)
- Block (16 bits)

Output:

- New CV (8 bits)

Uses:

- XOR
- Addition mod 256
- Bit rotation



Compression Function-round 1

- Step 1: $T1 = L \text{ XOR } K$
- Step 2: $T2 = T1 + CV \pmod{256}$
- Step 3: Rotate left by 3
- Step 4: $CV1 = \text{result XOR } R$

Compression Function-round 2

- Step 1: $T3 = R \text{ XOR } K$
- Step 2: $T4 = T3 + CV1 \pmod{256}$
- Step 3: Rotate left by 2
- Step 4: $CV2 = \text{result XOR } L$

Final Output:

- Hash = CV2

Example input

□ Message: 10101100

□ Key: 11001010

□ IV = Key

Padded block:

□ 10101100 0000 1000

Split:

□ L = 10101100

□ R = 00001000

Example-round1

- $T1 = 10101100 \text{ XOR } 11001010 = 01100110$
- $T2 = T1 + IV = 00110000$
- Rotate $\rightarrow 10000001$
- $CV1 = 10000001 \text{ XOR } 00001000 = 10001001$

Example-round2

- $T3 = 00001000 \text{ XOR } 11001010 = 11000010$
- $T4 = T3 + CV1 = 01001011$
- Rotate $\rightarrow 00101101$
- $CV2 = 00101101 \text{ XOR } 10101100 = 10000001$

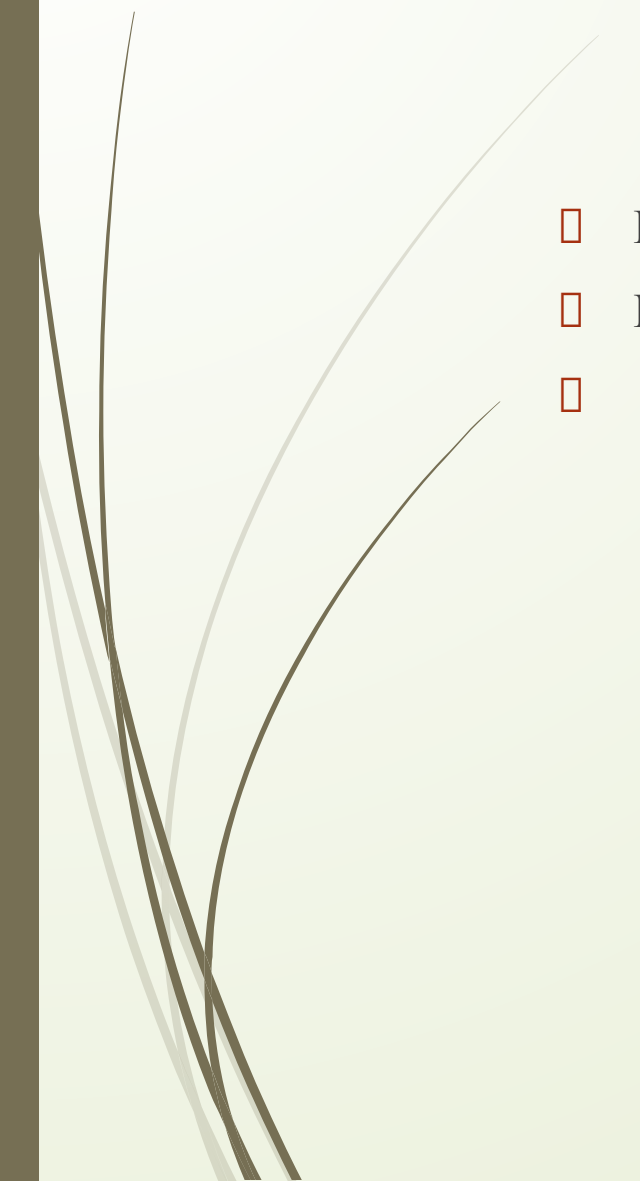


Final Hash Output

- ❑ **Final Hash:**
- ❑ 10000001
- ❑ **Size:**
- ❑ Exactly 8 bits
- ❑ **Depends on:**
- ❑ Message
- ❑ Secret Key



Security Features

- ❑ Key prevents forgery
 - ❑ Multiple rounds increase complexity
 - ❑ Chaining adds diffusion (small changes → large ~~ou~~put changes)
- 



Conclusion

- ❑ **The algorithm successfully:**
 - ❑ Uses a secret key
 - ❑ Applies Merkle–Damgård structure
 - ❑ Produces a fixed 8-bit hash
- ❑ **The output depends on both:**
 - ❑ Message
 - ❑ Key